

SecureMailScope

PROJECT OVERVIEW AND RISK CLASSIFICATION | Part 1 of 2

SecureMailScope is a local tool for auditing the security of email network traffic. Upload a PCAP/PCAPNG capture to examine SMTP, IMAP and POP3 connections for exposed authentication, weak encryption, STARTTLS problems and certificate issues. It presents evidence, risk findings, security scores and suggested fixes, with JSON, HTML and PDF reports. The frontend uses React/Vite; the backend uses Python/FastAPI.

How analysis works

Capture > packets > reconstructed TCP sessions > protocol/TLS/certificate checks > rules and ML > report. The system assesses sessions reconstructed from packets, providing context such as whether authentication occurred before encryption. Findings link to packet frames and timestamps; credentials are redacted.

How capture rules classify risk

Each rule has a predefined severity, rather than deriving severity from a percentage. All matching rules can trigger on the same session.

Severity	Conditions that trigger it
Critical	Authentication before TLS; plaintext commands after a STARTTLS request; obsolete SSL; ciphers containing RC4, DES, NULL or EXPORT.
High	TLS 1.0/1.1; static RSA key exchange without forward secrecy; expired or not-yet-valid certificates; hostname mismatch; weak certificate keys.
Medium	A lower TLS version selected than one offered; advertised STARTTLS unused in a complete session; failed chain validation or a self-issued certificate; SHA-1/MD5 signatures; unsuitable certificate extensions.
Low / Info	Supported categories, but current backend rules generate no findings at these levels. No findings means none observed, rather than guaranteed safety.

Certificate validity is checked at capture time. Weak keys include RSA below 2048 bits and elliptic curves outside the configured allowlist. A lower TLS version is only a downgrade indicator; failed chain validation does not establish maliciousness.

Score and ML classification

Every assessable session starts at **100; higher means better**. Each rule finding deducts **Critical 30, High 15, Medium 7, Low 2 or Info 0**.

A separate **Random Forest** predicts Critical/High/Medium/Low from **23 structured features**: TLS, cipher, certificate, authentication, session behavior and existing rule findings. Its prediction deducts an additional **20/10/5/0**, respectively. **Isolation Forest** flags unusual feature combinations; its anomaly flag does not directly deduct points.

Scores, modes and boundaries

PROJECT OVERVIEW AND RISK CLASSIFICATION | Continued

Example and aggregation

An expired certificate (-15), failed certificate chain (-7) and High ML prediction (-10) give **68/100**. Scores stop at zero. The project-wide score averages assessed sessions; incomplete sessions are excluded.

Browser simulation versus capture analysis

The separate **Browser simulation** uses predefined synthetic issues. Its session label is the highest severity present: **Critical > High > Medium**. No issues becomes Low; partial-only evidence becomes Unknown. It uses different deduction weights and no trained ML. Actual Capture analysis displays rule findings and ML risk separately.

Scope of the result

The project evaluates **observable email-transport security**. It does not decrypt emails or detect message-content threats, and its ML is trained on synthetic data. A high score means few observed problems; missing evidence, such as encrypted TLS 1.3 certificates, remains unknown.

Implementation references

Rule definitions: **backend/app/core/rule_engine.py**

Score calculation: **backend/app/core/posture_score.py**

ML features: **backend/app/ml/features.py**

Browser simulation: **src/engine.js**

Rule engine inputs

PARAMETERS AND CHECKS | Part 2 of 2

The rule engine takes a **reconstructed email session** as input. A certificate is the server's digital identity document: it binds identities to a public key and includes issuer details, validity dates and usage restrictions.

Input / parameter	Meaning and evaluation
Negotiated TLS version	The encryption protocol selected by the server. SSLv2/SSLv3: Critical. TLS 1.0/1.1: High.
Client-offered TLS versions	Versions supported by the client. Selecting a lower version than one offered: Medium, indicating a possible downgrade or server-policy limitation.
Negotiated cipher suite	Algorithms selected to secure the connection. RC4, DES (including 3DES), NULL or EXPORT: Critical.
Forward secrecy	Protection for recorded traffic if the server private key is later compromised. A cipher beginning with TLS_RSA_ uses static RSA: High.
STARTTLS advertised	Whether the server offered an upgrade from plaintext to TLS. Advertised but unused in a complete session: Medium.
STARTTLS used	Whether a TLS handshake was observed after the upgrade request. Evaluated alongside the advertised flag.
Plaintext after STARTTLS request	Client commands continuing without the expected encryption upgrade: Critical.
Authentication before TLS	AUTH, AUTHENTICATE, LOGIN or PASS before encryption: Critical. Credential values are redacted.
Certificate validity dates	Expired or not yet valid when the capture occurred: High.
Certificate hostname match	Whether the requested hostname from TLS SNI matches certificate SAN identities. Mismatch: High.

Certificate checks and evidence

PARAMETERS AND CHECKS | Continued

Input / parameter	Meaning and evaluation
Certificate trust chain	Whether the certificate validates through issuing authorities to a bundled trusted root. Validation failure, or matching subject and issuer treated as self-signed: Medium.
Certificate public key	RSA below 2048 bits, or an elliptic curve outside P-256/P-384/P-521: High.
Certificate signature algorithm	Hash used to sign the certificate. SHA-1 or MD5: Medium.
Certificate extensions	Requires Basic Constraints: CA=false and Extended Key Usage: serverAuth. Missing or unsuitable values: Medium.
Session completeness	Whether sufficient connection evidence was captured. Gates the STARTTLS-not-used rule; incomplete sessions receive no posture score.
Evidence metadata	Frame number, timestamp, stream ID and certificate SHA-256 fingerprint trace findings. They do not increase severity.

Missing evidence stays unknown. Unavailable certificates are skipped, missing SNI prevents hostname validation, and certificate revocation is not checked offline. ML features and numerical scores are evaluated separately from these deterministic rules.

Terms at a glance

TLS: Transport Layer Security.

STARTTLS / STLS: Commands to upgrade an existing plaintext connection to TLS.

SNI: Server Name Indication, carrying the requested server hostname.

SAN: Subject Alternative Name, listing identities covered by a certificate.

CA: Certificate Authority.

Forward secrecy: Protection of past session traffic against later compromise of a long-term private key.